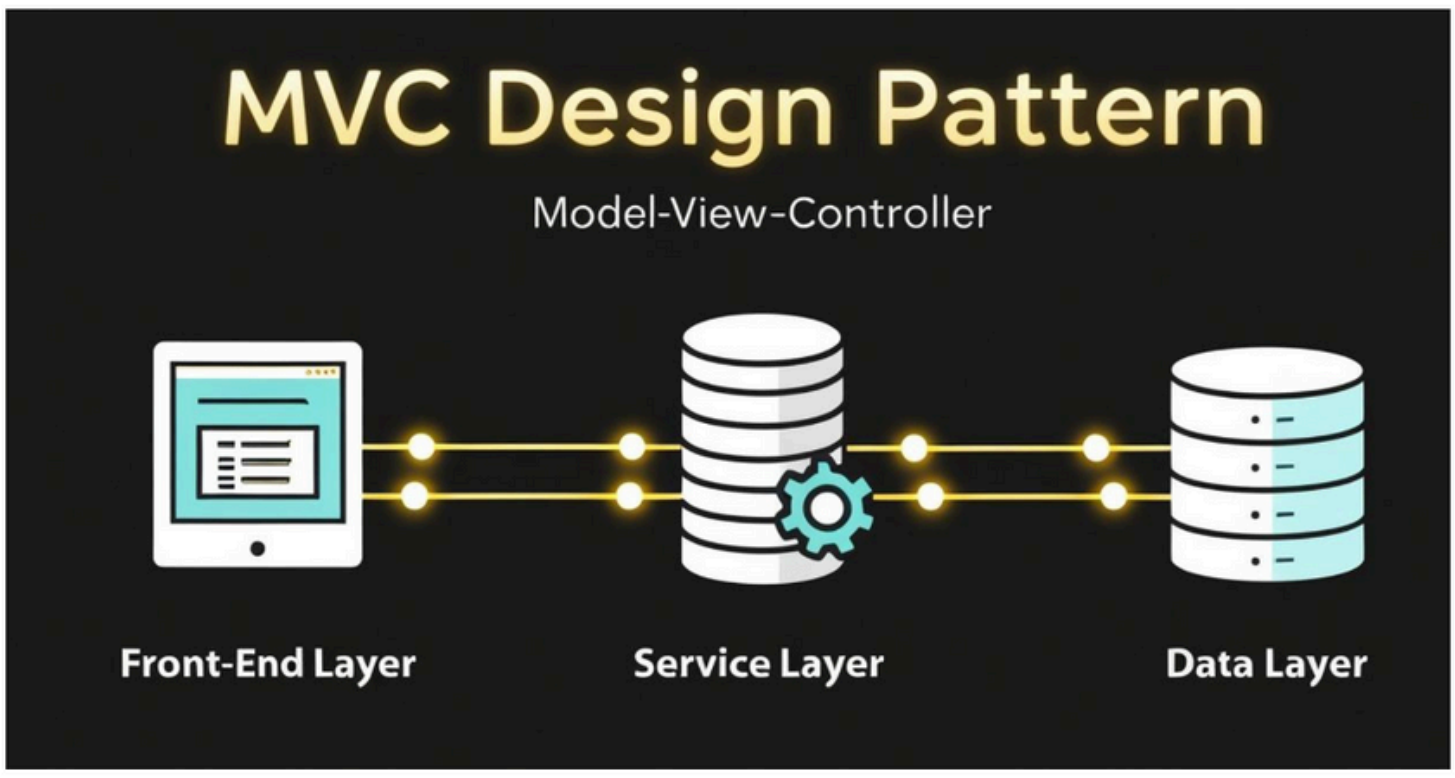
WHAT IS DEPENDENCY INJECTION?

What is Dependency Injection?

- Basically, it's a design pattern in Spring Boot that helps us to develop loosely coupled application
- Where one object supplies the required dependencies of another object
- It simplifies java object creation and management for developers



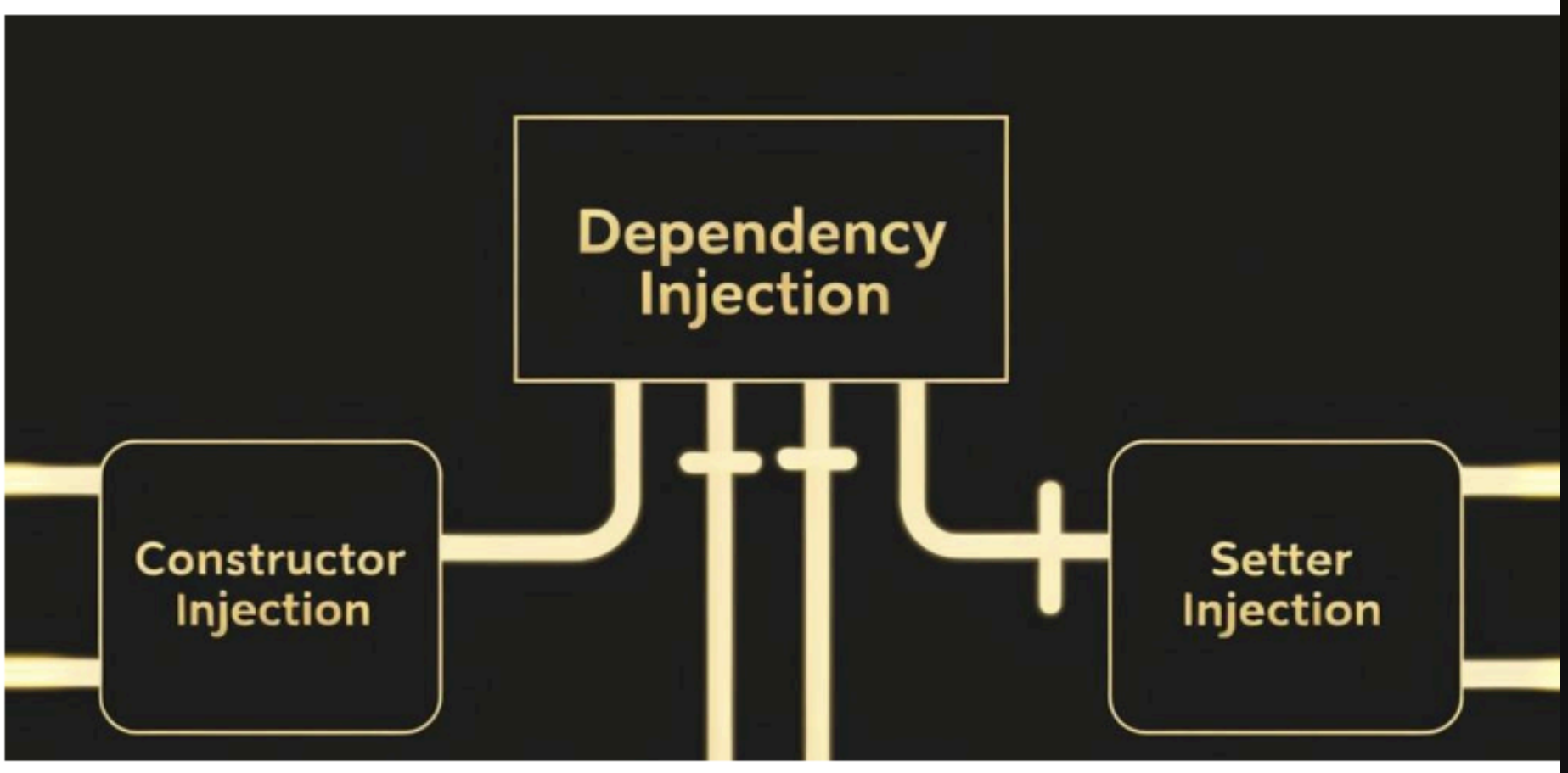
Example:

- Consider any web application based on the MVC design pattern
- Here, we follow a layered architecture, UI Layer, Service Layer & Data Layer
- The UI layer requires the Business layer object to display data to the user
- The Service layer depends on the Data layer → The Business layer requires a Database object
- The Spring container identifies the dependencies and injects them automatically – This process is called Dependency Injection

Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!

TYPES OF DEPENDENCY INJECTION (OR HOW MANY WAYS CAN WE INJECT A BEAN?)

Types of Dependency Injection (or How many ways can we inject a bean?)



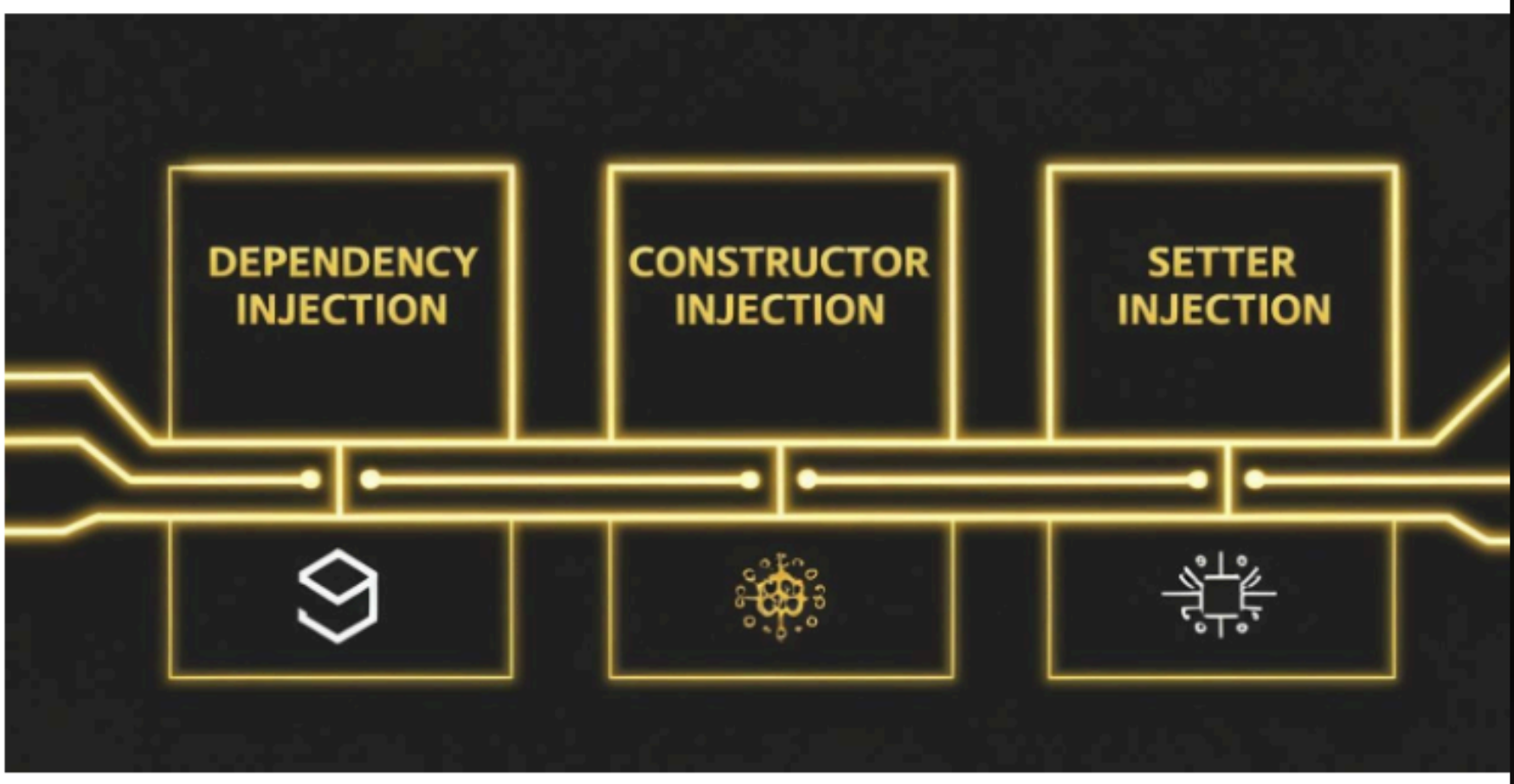
There are 2 types of Dependency Injection:

1. Constructor Injection
    - The Spring container injects dependencies as constructor arguments to achieve dependency injection.

2. Setter Injection
    - The Spring container injects dependencies as setter method arguments to achieve dependency injection.

WHEN TO USE WHICH?

When to Use Which?



Constructor Injection

- Case 1: If it's a mandatory dependency, Constructor Injection is recommended.
- Case 2: If you need to inject a dependency during object initialization, Constructor Injection is the best choice.

Disadvantage:

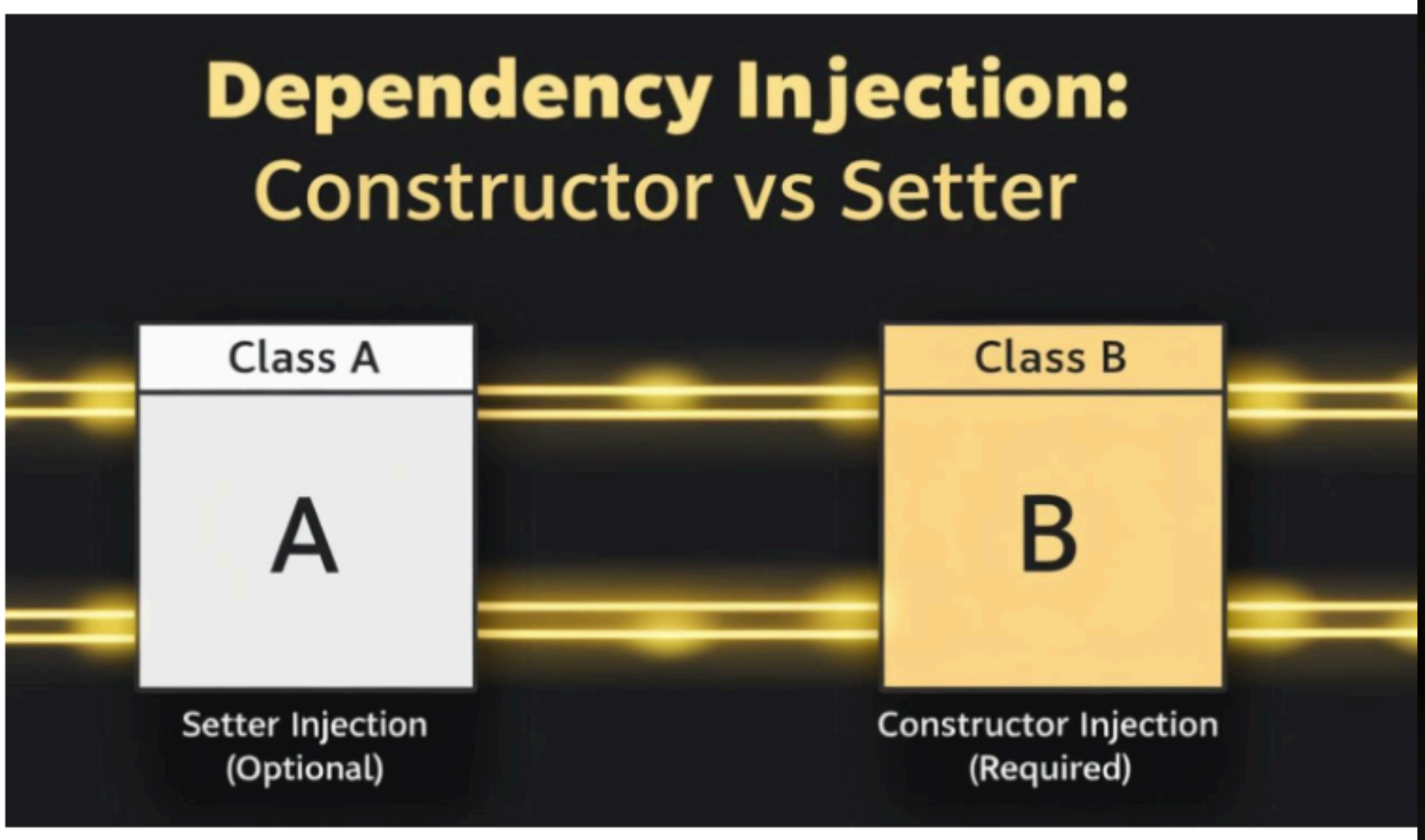
- If we pass more than 15 to 30 arguments in Constructor Injection, it becomes difficult to maintain.

Setter Injection

- Case 1: If it's an optional dependency, Setter Injection is recommended.
- Case 2: When you need to change dependencies at runtime, Setter Injection is preferred.

REAL TIME EXAMPLE:

Real Time Example:



Example #1:

- Assume we have two classes, Class A and Class B
- Both classes have some methods
- If Class A can work without Class B, then Setter Injection is fine.
- If Class A doesn't work without Class B, then we should use Constructor Injection.
- So, it depends on the use case!

Example #2: Constructor Injection

- Consider a Car class that requires an Engine
- The Car class should use Constructor Injection to ensure that these dependencies are provided during object creation

Example #3: Setter Injection

- Consider a Person class that can have an optional Address object
- The Person class can use Setter Injection to set the Address object after the Person object is created

PURPOSE OF THIS DOCUMENT

- **Use this PDF as a quick last-minute revision guide only.**
- **For more details, please perform R&D and gain a deeper understanding on each topic.**
- **Once the interview series is complete, we will cover this topic in detail.**